pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Case Study: Three Years of Curve and Spreads

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem definition: one table, then three years

The Desk Lead's Tenth Note

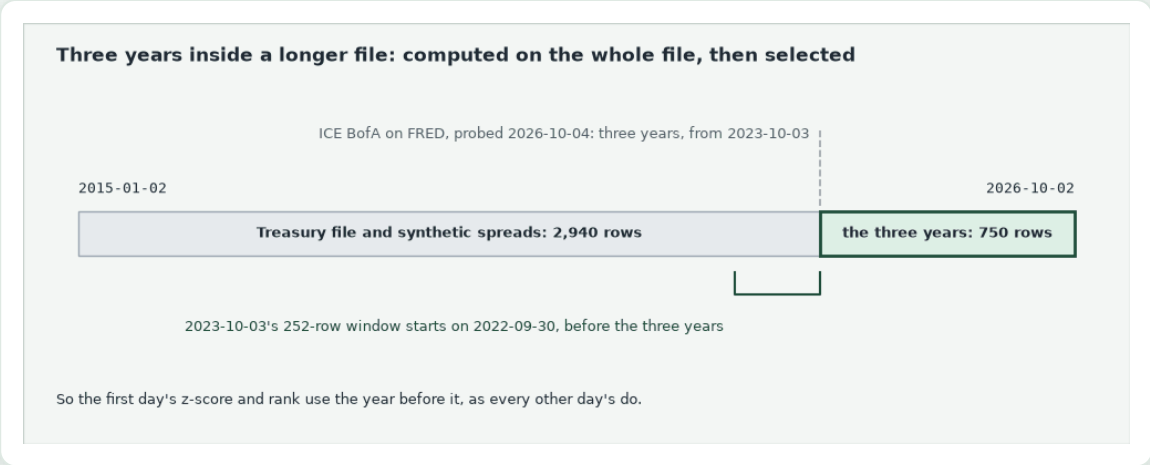
Week 2 ends with one table. For any date and any set of series, one row per series: the level, the move over a day, a week and a month in bp, and where it sits in its last year, tied to Excel and built from nothing after that date. Put it in "marketdata" with a test that proves it, and tag the week. Then use it: describe the curve and our two synthetic spreads over the three years FRED held of ICE's series when we checked on 4 October 2026, 3 October 2023 to 2 October 2026, on one four-panel page. With a key, show me ICE's own investment grade and high yield series arrived, in rows and dates only. Describe; no view on where anything goes.

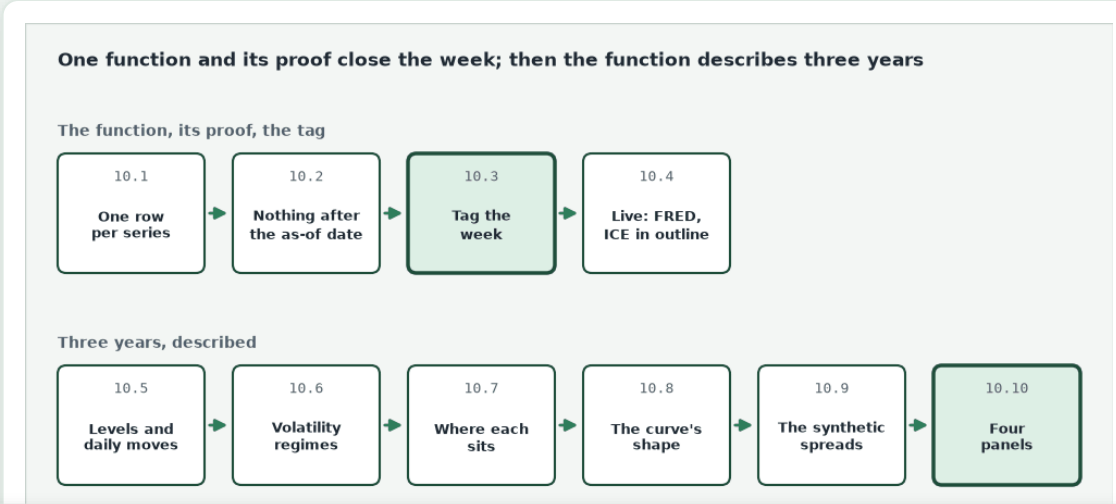
You are the desk's analyst: answer the note with your own "marketdata.py", and make every check an "assert".

The Data

File	What it holds	Units	Licence
`treasury_par_yields.csv`	Treasury par yields, 2,940 market days	percent	Public domain
`course3_synthetic_spreads.csv`	ig_spread_bp and hy_spread_bp on the same dates	bp	Synthetic: invented for this course, not ICE, Moody's, or any index

Three years inside a longer file, and how we will work





One function and its proof close the week; then the function describes three years.

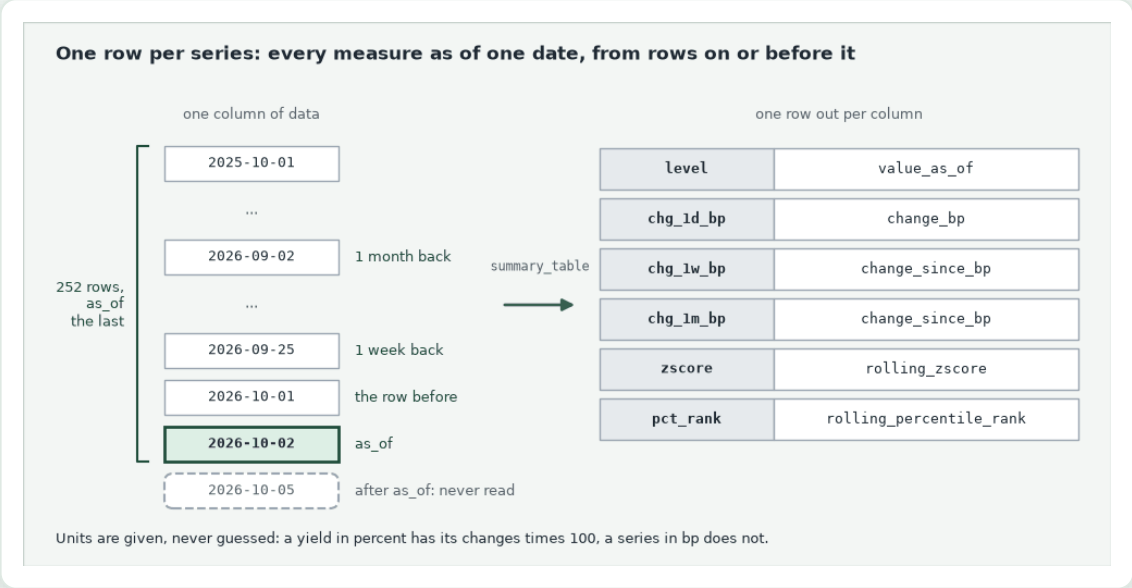
Today's Work Folder, and the Tests This Morning

```
Work folder pfi_course3_10_k4w7q2xr: 4 data files, Course 2's bondmath.py, marketdata.py as notebook 09 left it, 9 commits, tagged week1
........................................   [100%]
40 passed in 0.50s
```

10.1 One row per series: summary_table

Imports and the Two Files

```
import importlib
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import marketdata as md
sns.set_theme(style='whitegrid', palette=['#2E7D5B', '#9AA5B1', '#1E4D3A', '#52606D'])
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
spreads = pd.read_csv('course3_synthetic_spreads.csv', parse_dates=['date'], index_col='date',
                      float_precision='round_trip')
```



Every column is a function you already have: notebook 04's as-of value, notebook 06's changes, notebook 08's z-score and rank. The units are given, never guessed: a yield in percent needs its changes times 100, a spread in bp does not.

The docstring, then the function

```
%%writefile -a marketdata.py
def summary_table(data: pd.DataFrame, as_of, units: dict[str, str], window: int = 252) -> pd.DataFrame:
    """One row per column of `data`: the level and every measure as of one date, from rows on or before it.
    Units: units[column] is "pct" for a series in percent (a yield) or "bp" for a series in bp (a spread,
    slope, or fly). level is in that unit; chg_1d_bp, chg_1w_bp, chg_1m_bp are in bp; zscore has no unit;
    pct_rank is in percent.
    Convention: each column's missing values are dropped and only rows on or before as_of are used, so a later
    row can never change the table. level is the as-of value; chg_1d_bp is the change from the previous
    observation; chg_1w_bp and chg_1m_bp are change_since_bp from as_of - 7 days and as_of - 1 month;
    zscore and pct_rank are rolling_zscore and rolling_percentile_rank over `window` rows, today included.
    Raises ValueError if a column has no unit or a unit other than "pct" or "bp", or the index is not unique
    ascending dates.
    """
    _check_index(data.index, "data")
    as_of = pd.Timestamp(as_of)
    rows = {}
    for column in data.columns:
        unit = units.get(column)
        if unit not in ("pct", "bp"):
            raise ValueError(f"units[{column!r}] must be 'pct' or 'bp', not {unit!r}")
        percent_units = unit == "pct"
        series = data[column].loc[:as_of].dropna()
        _, level = value_as_of(series, as_of)
        rows[column] = {
            "unit": unit,
            "level": level,
            "chg_1d_bp": change_bp(series, 1, percent_units).iloc[-1],
            "chg_1w_bp": change_since_bp(series, as_of, as_of - pd.Timedelta(days=7), percent_units),
            "chg_1m_bp": change_since_bp(series, as_of, as_of - pd.DateOffset(months=1), percent_units),
            "zscore": rolling_zscore(series, window).iloc[-1],
            "pct_rank": rolling_percentile_rank(series, window).iloc[-1],
        }
    table = pd.DataFrame.from_dict(rows, orient="index")
    table.index.name = "series"
    return table
```

The 10 Yr and 2s10s on 2 October 2026

".loc[:as_of]" keeps every row up to and including that date's label. "DataFrame.from_dict(rows, orient="index")" makes each key of "rows" a row of the table.

The 10 Yr and 2s10s

```
pair = pd.DataFrame({'10 Yr': treasury['10 Yr'], '2s10s': md.slope_bp(treasury, '2 Yr', '10 Yr')})
pair_units = {'10 Yr': 'pct', '2s10s': 'bp'}
summary = md.summary_table(pair, '2026-10-02', pair_units)
summary.style.format('{:.2f}', subset=summary.columns[1:])
```

series	unit	level	chg_1d_bp	chg_1w_bp	chg_1m_bp	zscore	pct_rank
10 Yr	pct	5.28	4.00	11.00	49.00	3.08	99.60
2s10s	bp	45.00	-1.00	9.00	5.00	-0.51	28.17

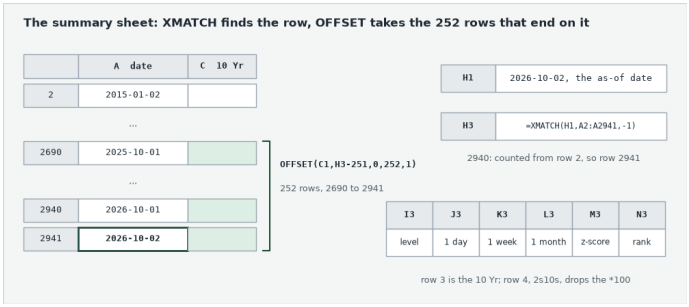
Against Excel's Reference Row

```
reference = pd.read_csv('course3_excel_reference.csv', parse_dates=['date'], index_col='date',
                        float_precision='round_trip').loc['2026-10-02']
checks = [('10 Yr', 'chg_1d_bp', 'chg_10y_bp'), ('10 Yr', 'chg_1m_bp', 'chg1m_10y_bp'),
          ('10 Yr', 'zscore', 'zscore252_10y'), ('10 Yr', 'pct_rank', 'pctrank252_10y'),
          ('2s10s', 'level', 'slope_2s10s_bp'), ('2s10s', 'zscore', 'zscore252_2s10s'),
          ('2s10s', 'pct_rank', 'pctrank252_2s10s')]
gaps = [abs(summary.loc[row, ours] - reference[excel]) for row, ours, excel in checks]
assert max(gaps) < 1e-9, 'a value differs from Excel'
print(f'{len(gaps)} values, largest gap {max(gaps):.1e}')
7 values, largest gap 4.6e-13
```

Excel side by side: XMATCH, XLOOKUP and OFFSET

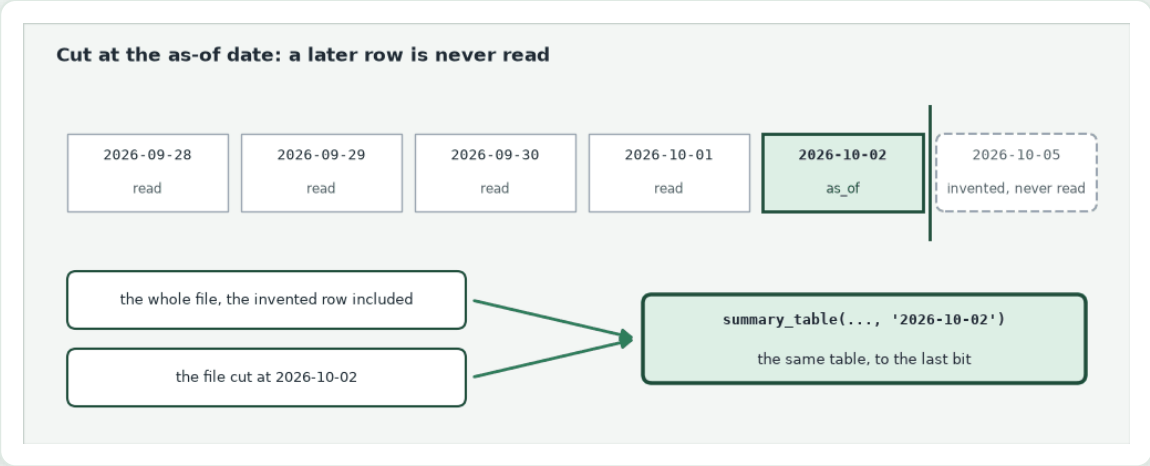
Excel side by side. A sheet with dates in A2:A2941, the 2 Yr in B, the 10 Yr in C and 2s10s in D as "=(C2-B2)*100"; the as-of date in H1, the 10 Yr's row 3. "XMATCH" with -1 gives the position of the date, or of the last date before it, as notebook 04's "XLOOKUP" match mode -1 does.

Job	Python	Excel	Result
Position, H3	`value_as_of`	`=XMATCH($H$1,$A$2:$A$2941,-1)`	2940
Level	`summary.loc['10 Yr', 'level']`	`=XLOOKUP($H$1,$A$2:$A$2941,C$2:C$2941,,-1)`	5.28
1 day	`chg_1d_bp`	`=(I3-INDEX(C$2:C$2941,H3-1))*100`	4.0000000000000036
1 week	`chg_1w_bp`	`=(I3-XLOOKUP($H$1-7,$A$2:$A$2941,C$2:C$2941,,-1))*100`	11.000000000000032
1 month	`chg_1m_bp`	`=(I3-XLOOKUP(EDATE($H$1,-1),$A$2:$A$2941,C$2:C$2941,,-1))*100`	49.00000000000002
Z-score	`zscore`	`=STANDARDIZE(I3,AVERAGE(OFFSET(C$1,H3-251,0,252,1)),STDEV.S(OFFSET(C$1,H3-251,0,252,1)))`	3.075234405274332
Rank	`pct_rank`	`=COUNTIF(OFFSET(C$1,H3-251,0,252,1),"<="&I3)/COUNT(OFFSET(C$1,H3-251,0,252,1))*100`	99.60317460317461
Rows in the window	`window=252`	`=ROWS(OFFSET(C$1,H3-251,0,252,1))`	252



"summary_table" puts the 10 Yr and 2s10s on 2 October 2026 in one row each, equal to Excel's reference row within 4.6e-13.

10.2 Nothing after the as-of date



The note asked for a table "built from nothing after that date". A row after the as-of date must change nothing.

Q. Add a made-up row on Monday 2026-10-05. Does the 2 October table move?

```
invented = pd.DataFrame({'10 Yr': [9.99], '2s10s': [500.0]},
                        index=pd.DatetimeIndex(['2026-10-05'], name='date'))
pair_plus = pd.concat([pair, invented])
print(pair_plus.tail(3))
   10 Yr  2s10s
date
2026-10-01   5.24   46.0
2026-10-02   5.28   45.0
2026-10-05   9.99  500.0
```

"pd.DatetimeIndex([...], name='date')" makes a one-date index named like the file's.

```
pd.testing.assert_frame_equal(md.summary_table(pair_plus, '2026-10-02', pair_units), summary,
                              check_exact=True)
seen = md.summary_table(pair_plus, '2026-10-05', pair_units).loc['10 Yr', 'chg_1d_bp']
print(f'As of 2026-10-02 nothing moved; as of 2026-10-05 the row is read: {seen:+.1f} bp')
As of 2026-10-02 nothing moved; as of 2026-10-05 the row is read: +471.0 bp
```

The shortcut, a holiday and a month end

Q. Why not just take the last row?

```
shortcut_level = pair['10 Yr'].iloc[-1]
as_of_level = md.summary_table(pair, '2026-09-30', pair_units).loc['10 Yr', 'level']
print(f'As of 2026-09-30: the last row says {shortcut_level}, summary_table says {as_of_level}')
As of 2026-09-30: the last row says 5.28, summary_table says 5.29
```

The last row is right only when the as-of date is the file's last day. Q. Labor Day, 7 September 2026, has no row. What does the table use?

```
holiday_table = md.summary_table(pair, '2026-09-07', pair_units)
month_end_table = md.summary_table(pair, '2026-09-30', pair_units)
pd.DataFrame({'2026-09-07': holiday_table.loc['10 Yr'],
              '2026-09-30': month_end_table.loc['10 Yr']})
```

	2026-09-07	2026-09-30
unit	pct	pct
level	4.78	5.29
chg_1d_bp	1.0	3.0
chg_1w_bp	3.0	18.0
chg_1m_bp	13.0	56.0
zscore	2.121722	3.252356
pct_rank	99.206349	100.0

On the holiday the level is Friday 4 September's 4.78, as notebook 04's "value_as_of" falls back.

Excel side by side. H1 set to each date, the rest of the sheet unchanged.

Job	Python	Excel	Result
Position on the holiday	`value_as_of`	`=XMATCH($H$1,$A$2:$A$2941,-1), H1 2026-09-07`	2921, Friday's row
Level on the holiday	`holiday_table.loc['10 Yr', 'level']`	`=XLOOKUP($H$1,$A$2:$A$2941,C$2:C$2941,,-1)`	4.78
Level at the month end	`month_end_table.loc['10 Yr', 'level']`	`=XLOOKUP(...)`	5.29

On all three dates, the sheet's 36 values equal the function's within 5.5e-13.

Two tests

Q. Add a known-date test and a no-lookahead test.

```
%%writefile -a test_marketdata.py
def ten_year_frame() -> tuple[pd.DataFrame, dict[str, str]]:
    curve = read_treasury()
    data = pd.DataFrame({"10 Yr": curve["10 Yr"], "2s10s": md.slope_bp(curve, "2 Yr", "10 Yr")})
    return data, {"10 Yr": "pct", "2s10s": "bp"}
def test_summary_table_known_date():
    data, units = ten_year_frame()
    reference = read_reference().loc["2026-10-02"]
    table = md.summary_table(data, "2026-10-02", units)
    ten, slope = table.loc["10 Yr"], table.loc["2s10s"]
    assert ten["unit"] == "pct" and slope["unit"] == "bp"
    assert ten["level"] == 5.28
    assert ten["chg_1d_bp"] == pytest.approx(reference["chg_10y_bp"], abs=1e-9)
    assert ten["chg_1m_bp"] == pytest.approx(reference["chg1m_10y_bp"], abs=1e-9)
    expected_week = (data.loc["2026-10-02", "10 Yr"] - data.loc["2026-09-25", "10 Yr"]) * 100
    assert ten["chg_1w_bp"] == pytest.approx(expected_week, abs=1e-9)
    assert ten["zscore"] == pytest.approx(reference["zscore252_10y"], abs=1e-9)
    assert ten["pct_rank"] == pytest.approx(reference["pctrank252_10y"], abs=1e-12)
    assert slope["level"] == pytest.approx(reference["slope_2s10s_bp"], abs=1e-9)
    assert slope["zscore"] == pytest.approx(reference["zscore252_2s10s"], abs=1e-9)
    assert slope["pct_rank"] == pytest.approx(reference["pctrank252_2s10s"], abs=1e-12)
def test_summary_table_no_lookahead():
    data, units = ten_year_frame()
    for cut in CUT_DATES:
        full = md.summary_table(data, cut, units)
        cut_only = md.summary_table(data.loc[:cut], cut, units)
        pd.testing.assert_frame_equal(full, cut_only, check_exact=True)
```

```
.....................................   [100%]
42 passed in 0.50s
```

The second applies notebook 07's rule to a whole table: at each of "CUT_DATES", the table from the whole file equals the table from the file cut there.

A row after the as-of date changes nothing, and a holiday falls back to Friday, as Excel's -1 match does: 42 passed.

10.3 Tag the week

Read the Diff, Commit

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
subject = 'Notebook 10: summary_table'
body = 'Week 2 ends with one table built only from rows on or before its as-of date.'
!git -C "{work}" add marketdata.py test_marketdata.py
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
 M marketdata.py
 M test_marketdata.py
 marketdata.py    | 40 ++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 35 ++++++++++++++++++++++++++++++++++
 2 files changed, 75 insertions(+)
```

Between the Tags

```
!git -C "{work}" diff --stat week1 week2
!git -C "{work}" log --oneline week1..week2
 marketdata.py    | 215 ++++++++++++++++++++++++++++++++++++++++++++++++++
 slopes_2026.csv    | 191 ++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 229 ++++++++++++++++++++++++++++++++++++++++++++++++++++
 3 files changed, 635 insertions(+)
51b40fc Notebook 10: summary_table
19940f9 Notebook 09: slope_bp, butterfly_bp, fly_weights, and slopes_2026.csv
de7069f Notebook 08: rolling_zscore and rolling_percentile_rank
85a1fa6 Notebook 07: rolling_vol_bp and assert_no_lookahead
1f5838a Notebook 06: change_bp, return_pct, and change_since_bp
```

Name the Week

```
!git -C "{work}" tag week2
!git -C "{work}" log --oneline --decorate
51b40fc (HEAD -> main, tag: week2) Notebook 10: summary_table
19940f9 Notebook 09: slope_bp, butterfly_bp, fly_weights, and slopes_2026.csv
de7069f Notebook 08: rolling_zscore and rolling_percentile_rank
85a1fa6 Notebook 07: rolling_vol_bp and assert_no_lookahead
1f5838a Notebook 06: change_bp, return_pct, and change_since_bp
ace289a (tag: week1) Notebook 05: write_cache, read_cache, and cached_fred_series
9f01b1c Notebook 04: last_per_period and value_as_of
a439502 Notebook 03: missing_weekdays, align, and stale_runs
ceea769 Notebook 02: fred_params, parse_fred_observations, and fred_series
a28686d Notebook 01: get_secret, a .gitignore, and .env.example
```

"week2" names the module as week 2 ends; between the tags lie notebooks 06 to 10, a data file among them. Now the function earns its keep.

10.4 Live: the same table from FRED

This product uses the FRED® API but is not endorsed or certified by the Federal Reserve Bank of St. Louis.

Why three years: since April 2026 ICE BofA's series on FRED hold only three years of observations, as notebook 05's story told (source https://fred.stlouisfed.org/series/BAMLC0A0CM, read 2026-10-04). When the course checked on 2026-10-04, they began on 2023-10-03, so the case study takes that span from the two files.

The Key Cell

```
from pathlib import Path
ENV_FILE = Path.home() / "projects" / "my_bondmath" / ".env"
NO_KEY_MESSAGE = "Live cell skipped: no FRED key was found. Every other cell runs without one."
SHOW_LICENSED_VALUES = False
```

```
try:
    fred_key = md.get_secret("FRED_API_KEY", env_file=ENV_FILE)
    print("FRED key: found. It is held in memory and is never printed.")
except RuntimeError as error:
    fred_key = None
    print(f"FRED key: not found. Live cells will be skipped. {error}")
```

```
FRED key: not found. Live cells will be skipped. Secret FRED_API_KEY not found. Looked in: the environment variable FRED_API_KEY, Colab's Secrets panel, and the file .env given as env_file.
```

DGS through summary_table, and ICE in outline

DGS2 and DGS10 Through summary_table

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    live_2y = md.fred_series('DGS2', fred_key, start='2023-10-03')
    live_10y = md.fred_series('DGS10', fred_key, start='2023-10-03')
    live_curve = pd.DataFrame({'2 Yr': live_2y, '10 Yr': live_10y}).dropna()
    live_pair = pd.DataFrame({'10 Yr': live_curve['10 Yr'],
                              '2s10s': md.slope_bp(live_curve, '2 Yr', '10 Yr')})
    live_table = md.summary_table(live_pair, '2026-10-02', pair_units)
    pd.testing.assert_frame_equal(live_table, summary, check_exact=False, rtol=0, atol=1e-9)
    print(f'{len(live_curve)} days from FRED, the last {live_curve.index[-1]:%Y-%m-%d}; '
          f'the table as of 2026-10-02 equals 10.1 within 1e-9')
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

ICE's Two Series, Rows and Dates

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    for live_id in ['BAMLC0A0CM', 'BAMLH0A0HYM2']:
        live_ice = md.fred_series(live_id, fred_key, start='2023-10-03')
        live_dots = live_ice.isna().sum()
        live_weekend = (live_ice.index.dayofweek >= 5).sum()
        assert live_ice.index.is_unique and live_ice.index.is_monotonic_increasing, live_id
        print(f'{live_id}: {len(live_ice)} rows, {live_ice.index[0]:%Y-%m-%d} to '
              f'{live_ice.index[-1]:%Y-%m-%d}, {live_dots} . rows, {live_weekend} weekend rows')
        if SHOW_LICENSED_VALUES:
            live_ice_bp = pd.DataFrame({live_id: live_ice * 100})
            print(md.summary_table(live_ice_bp, live_ice_bp.index[-1], {live_id: 'bp'}).round(2))
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

With Your Key

The first cell prints FRED's row count, its last date and the check line, since FRED's "DGS" series are Treasury's own numbers. The second prints rows, dates, "." rows and weekend rows for each series, and values only with "SHOW_LICENSED_VALUES = True" on your own machine. Each series' first date is later than 2023-10-03 on any day after 2026-10-04, since FRED keeps only the last three years.

With a key the same table comes from FRED, and ICE's two series arrive as rows and dates. Every number from here on is the Treasury file or the synthetic spreads.

10.5 Three years of levels and daily moves

Q. Put five yields, three slopes, two flies and the two synthetic spreads side by side.

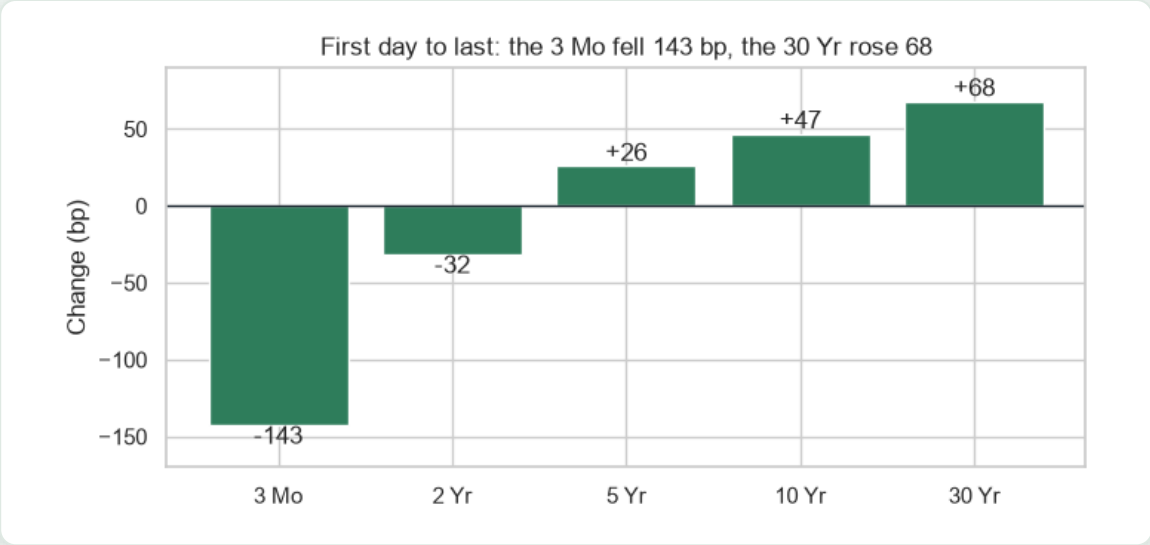
```
YIELDS = ['3 Mo', '2 Yr', '5 Yr', '10 Yr', '30 Yr']
SHAPES = ['2s10s', '5s30s', '3m10y', '2s5s10s', '5s10s30s']
market = treasury[YIELDS].copy()
market['2s10s'] = md.slope_bp(treasury, '2 Yr', '10 Yr')
market['5s30s'] = md.slope_bp(treasury, '5 Yr', '30 Yr')
market['3m10y'] = md.slope_bp(treasury, '3 Mo', '10 Yr')
```

```
market['2s5s10s'] = md.butterfly_bp(treasury, '2 Yr', '5 Yr', '10 Yr')
market['5s10s30s'] = md.butterfly_bp(treasury, '5 Yr', '10 Yr', '30 Yr')
market = market.join(spreads)
units = {name: 'pct' for name in YIELDS} | {name: 'bp' for name in market.columns[5:]}
print(f'{market.shape[1]} series on {len(market):,} market days')
12 series on 2,940 market days
```

Q. Select the three years. Is every value there?

```
three_years = market.loc['2023-10-03':'2026-10-02']
assert len(three_years) == 750 and three_years.notna().all().all(), 'a row or a value is missing'
assert (three_years.index.dayofweek >= 5).sum() == 0, 'a weekend date'
print(f'{len(three_years)} market days, {three_years.shape[1]} series, no blank, no weekend')
750 market days, 12 series, no blank, no weekend
```

First day to last, and the daily moves




```
daily = pd.DataFrame({name: md.change_bp(market[name], 1, units[name] == 'pct')
                      for name in market.columns})
daily_3y = daily.loc['2023-10-03':'2026-10-02']
moved = daily_3y[YIELDS].round(6)
```

Excel side by side. A second sheet holds the three years: row 2 is 2023-10-02, and rows 3 to 752 are the window. Column A holds the date, B the 2 Yr, C the 10 Yr, D and E the synthetic spreads, with H3 "=(C3-C2)*100" and I3 "=(B3-B2)*100" filled down.

Job	Python	Excel	Result
Largest 2 Yr fall	`moves.loc['2 Yr', 'largest_fall_bp']`	`=MIN(I3:I752)`	-28.000000000000025
Its date	`moves.loc['2 Yr', 'fall_date']`	`=XLOOKUP(MIN(I3:I752),I3:I752,A3:A752)`	45506, 2024-08-02 as a date

The file holds each move's size and date, never its reason, so none is given.

10.6 Volatility regimes

Notebook 07's 63-row volatility moves slowly enough to show stretches of calm and of large moves. A regime here is such a stretch, told by its size and dates only.

Lows, Highs, and the Last Day

```
regimes = pd.DataFrame({'low': vol_63.min(), 'low_date': vol_63.idxmin().dt.date,
                        'high': vol_63.max(), 'high_date': vol_63.idxmax().dt.date,
                        'last': vol_63.iloc[-1]})
regimes.round(2)
```

	low	low_date	high	high_date	last
10 Yr	3.11	2026-02-03	8.25	2023-12-20	4.63
ig_spread_bp	0.81	2025-03-20	1.99	2025-06-17	1.09
hy_spread_bp	3.22	2025-12-24	8.27	2025-06-11	5.41

The 2 Yr's row waits for Predict 3.

10.7 Where each series sits on 2 October 2026

The note's table, on every series. Q. Run "summary_table" on all twelve series.

```
today = md.summary_table(market, '2026-10-02', units)
(today.style.format('{:.2f}', subset=today.columns[1:])
    .background_gradient(cmap='Greens', subset=['pct_rank']))
```

series	unit	level	chg_1d_bp	chg_1w_bp	chg_1m_bp	zscore	pct_rank
3 Mo	pct	4.19	2.00	-5.00	27.00	2.58	98.02
2 Yr	pct	4.83	5.00	2.00	44.00	2.51	98.02
5 Yr	pct	5.06	5.00	8.00	52.00	2.80	99.60
10 Yr	pct	5.28	4.00	11.00	49.00	3.08	99.60
30 Yr	pct	5.63	2.00	14.00	36.00	3.08	99.60
2s10s	bp	45.00	-1.00	9.00	5.00	-0.51	28.17
5s30s	bp	57.00	-3.00	6.00	-16.00	-2.07	5.95
3m10y	bp	109.00	2.00	16.00	22.00	2.07	100.00
2s5s10s	bp	1.00	1.00	3.00	11.00	2.70	98.81
5s10s30s	bp	-13.00	1.00	0.00	10.00	2.58	100.00
ig_spread_bp	bp	97.00	-1.00	1.10	1.90	0.22	61.11
hy_spread_bp	bp	328.70	-1.30	11.20	0.10	0.90	73.81

A rank compares a day with its own past; it says nothing about what comes next.

10.8 The shape of the curve

Notebook 09's signs: a slope is the long yield less the short, and a fly is twice the belly less both wings.

The Last Day Below Zero

```
for name in ['2s10s', '3m10y']:
    print(f'{name}: last below zero on {shape[shape[name] < 0].index[-1]:%Y-%m-%d}')
2s10s: last below zero on 2024-09-05
3m10y: last below zero on 2025-10-16
```

Excel side by side. On the three-year sheet, J3 "=(C3-B3)*100" filled down.

Job	Python	Excel	Result
2s10s days below zero	`signs.loc['2s10s', 'below_zero']`	`=COUNTIF(J3:J752,"<0")`	227

10.9 The synthetic spreads

Synthetic: invented by the course's generator, not ICE, Moody's, or any index.

Range and Gap

```
synthetic = three_years[['ig_spread_bp', 'hy_spread_bp']]
gap = synthetic['hy_spread_bp'] - synthetic['ig_spread_bp']
spread_stats = pd.DataFrame({'first': synthetic.iloc[0], 'last': synthetic.iloc[-1],
                             'low': synthetic.min(), 'high': synthetic.max()})
spread_stats.loc['gap'] = [gap.iloc[0], gap.iloc[-1], gap.min(), gap.max()]
spread_stats.round(1)
```

	first	last	low	high
ig_spread_bp	93.5	97.0	80.7	114.2
hy_spread_bp	234.3	328.7	198.7	394.3
gap	140.8	231.7	118.0	281.4

Excel side by side. On the three-year sheet, F3 "=D3-D2", G3 "=E3-E2" and K3 "=E3-D3", filled down. "CORREL" is the same Pearson correlation as "corr".

Job	Python	Excel	Result
Narrowest gap	`gap.min()`	`=MIN(K3:K752)`	117.99999999999999
Widest gap	`gap.max()`	`=MAX(K3:K752)`	281.4

10.10 One page, four panels

Q. Put the three years on one page.

```
fig, axes = plt.subplots(2, 2, sharex=True, figsize=(12, 7))
five = ['#9AA5B1', '#52606D', '#2E7D5B', '#1E4D3A', '#1F2933']
three_years[YIELDS].plot(ax=axes[0, 0], title='Yields (percent)', color=five)
three_years[SHAPES].plot(ax=axes[0, 1], title='Slopes and flies (bp)', color=five)
vol_63.plot(ax=axes[1, 0], title='63-row volatility (bp a day, spreads synthetic)')
synthetic.plot(ax=axes[1, 1], title='Spreads (bp, synthetic)')
for panel in axes.flat:
    panel.legend(fontsize=7, ncols=2)
fig.suptitle('3 October 2023 to 2 October 2026: the Treasury file and the synthetic spreads')
plt.show()
```

None says why a series moved, what it does next, or what anyone should do.

One page, four panels, six observations, every number from a table above: the note is answered.

Summary: the Checks Behind Every Answer

Check	Result
summary_table against Excel	36 values on three dates within 5.5e-13
Nothing after the as-of date	an invented row changes nothing; 42 passed
The window	750 market days, no blank, no weekend
2s10s below zero	227 days, as COUNTIF
Synthetic label	on every chart and panel that shows the spreads

Bring it home, and what you can now do

Copy the two files into your own folder, test, commit, and tag the week there. In a PowerShell terminal in VS Code with the course's ".venv" active, use the work folder name your setup cell printed:

```
$work = "$env:TEMP\pfi_course3_10_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 10: summary_table"
git tag week2
git diff --stat week1 week2
```

The copy replaces your two files, so anything of your own goes into the work folder first. The data files the tests read are already there, and pytest says "42 passed", more with your exercise tests. In Colab, download the two files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Build one row per series of every measure as of one date with "summary_table".

Prove a table reads nothing after its date, with an invented row and a test.

Tie a table to an Excel sheet built with "XMATCH", "XLOOKUP" and "OFFSET".

Tag the end of a week and read what changed between two tags.

Describe three years of a curve and two spreads in numbers and dates, with no forecast.

Predict first: three lows, a typical day, a busy year

Predict 1

Over the three years the 5 Yr, 10 Yr and 30 Yr each touched a low. Those three lows fell on:

A) three different days

B) the same day

C) two the same, one apart

```
yields_3y = three_years[YIELDS]
levels = pd.DataFrame({'first': yields_3y.iloc[0], 'last': yields_3y.iloc[-1],
                       'low': yields_3y.min(), 'low_date': yields_3y.idxmin().dt.date,
                       'high': yields_3y.max(), 'high_date': yields_3y.idxmax().dt.date})
levels['moved_bp'] = (levels['last'] - levels['first']) * 100
levels.round(2)
```

Predict 2

Ignore the sign: over 750 days, how far did the 10 Yr move on an average day, in bp? Write down a number.

```
moves = pd.DataFrame({'largest_rise_bp': moved.max(), 'rise_date': moved.idxmax().dt.date,
                      'largest_fall_bp': moved.min(), 'fall_date': moved.idxmin().dt.date,
                      'mean_abs_bp': daily_3y[YIELDS].abs().mean()})
moves.round(3)
```

Predict 3

The 2 Yr's 63-row volatility, averaged by calendar year: which year is highest?

A) 2023's last quarter

B) 2024

C) 2026

```
VOL_SERIES = ['2 Yr', '10 Yr', 'ig_spread_bp', 'hy_spread_bp']
vol_63 = pd.DataFrame({name: md.rolling_vol_bp(daily[name].dropna(), 63)
                       for name in VOL_SERIES}).loc['2023-10-03':]
yearly_vol = vol_63.groupby(vol_63.index.year).mean()
yearly_vol.round(2)
```

Predict first: the first day, the long fly, the two spreads

Predict 4

On 2023-10-03, the first day of the three years, how many of the five yields sat at a percentile rank of 100 in their own last 252 market days? Write down a number.

```
first_day = md.summary_table(market, '2023-10-03', units)
print(f"Yields at a rank of 100: {(first_day.loc[YIELDS, 'pct_rank'] == 100).sum()} of 5")
```

Predict 5

On how many of the 750 days was the 5s10s30s fly above zero?

A) none

B) about half

C) most

```
shape = three_years[SHAPES].round(6)
signs = pd.DataFrame({'below_zero': (shape < 0).sum(), 'zero': (shape == 0).sum(),
                      'above_zero': (shape > 0).sum()})
signs
```

Predict 6

The generator builds each high yield shock from the investment grade shock plus noise of its own. The correlation of the two daily changes over the three years is nearer:

A) 0

B) 0.5

C) 1

```
correlation = daily_3y['ig_spread_bp'].corr(daily_3y['hy_spread_bp'])
same_way = (daily_3y['ig_spread_bp'] * daily_3y['hy_spread_bp'] > 0).mean()
print(f'correlation {correlation:.4f}; '
      f'both moved the same way on {same_way * 100:.1f} percent of days')
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "return known.iloc[-(window + 1):]" keeps one row too many: the window does not hold 252 rows.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
One row per series, tied to Excel	summary_table, 36 sheet values within 5.5e-13
Nothing after its date	Two tests, 42 passed
The week tagged	week2, notebooks 06 to 10 between the tags
Three years described	Six observations, every one a number and a date
ICE's two series	With your key, rows and dates only

Be proud: a description that stays inside its data is harder to write than a story, and a desk can rely on it.

Next, SQL With sqlite3 (notebook 11) moves the course's holdings and market data into a database, every query checked against pandas.